

10 — Sincronização, Barreiras e `__syncthreads()`

Pré-requisito: [02 — blocks](#), [07 — shared](#) e [09 — divergência](#).

1. O que a barreira garante

`__syncthreads()` é uma barreira de **thread block**:

1. cada thread não-exitada do block precisa alcançar a mesma barreira;
2. quem chega cedo espera;
3. acessos global/shared anteriores das threads participantes ficam ordenados/visíveis antes de elas prosseguirem.

Ela não sincroniza blocks comuns, grids diferentes nem host. Também não “espera só o warp”.

```
shared[tid] = input[i];
__syncthreads();
output[i] = shared[neighbor];
```

A barreira separa produção e consumo cooperativo.

2. O que acontece com warps

Quando um warp alcança a barreira, suas threads participantes deixam de prosseguir até o block satisfazer a condição. O scheduler pode executar outros warps ready, inclusive de outros blocks residentes. Se não houver outro trabalho elegível, unidades ficam ociosas.

3. Barreiras em controle não uniforme

Isto é incorreto:

```
if (threadIdx.x < 16) __syncthreads();
```

Parte do block espera por threads que nunca chegam. Uma barreira em condicional só é segura quando a condição é uniforme para todo o block participante.

`__syncwarp(mask)` tem escopo de warp e deve ser usado com máscara correta quando a comunicação é intra-warp. Não substitua uma barreira de block sem provar o escopo.

Pergunta rápida

Todas as threads escrevem e depois leem apenas seu próprio índice em shared memory. A barreira é necessária para correção?

► Resposta

O que guardar

- `__syncthreads()` = encontro + ordenação/visibilidade dentro de um block.
- Warp que chega espera; o SM tenta emitir outro warp.
- Barreira em caminho não uniforme pode travar ou produzir comportamento inválido.

Conexão: atomics resolvem outro problema — atualização indivisível — e não substituem automaticamente barreiras.

Fonte primária: [CUDA Programming Guide — Synchronization Functions](#).